

SEMANA 2 — DÍA 3 (REESTRUCTURADO Y AUTOGESTIVO)

Miércoles 15 de abril – Organización del Proyecto (carpetas, módulos, os/sys)

Objetivo del día:

Que cada equipo organice su proyecto en carpetas y módulos, y comprenda cómo dividir un script grande en funciones pequeñas y reutilizables.

Al final de la clase, deben tener **la estructura completa del proyecto y módulos base creados**, listos para integrar el Script 1 v0 mañana.

TEORÍA

1. ¿Por qué dividir un proyecto en módulos?

Un solo archivo con 200 líneas es difícil de:

- leer
- depurar
- mejorar
- trabajar en equipo

Dividirlo en módulos permite:

- separar responsabilidades
- reutilizar funciones
- evitar duplicar código
- trabajar en paralelo

Idea clave:

Un módulo = un archivo .py que contiene funciones relacionadas con un solo tema.

2. Estructura recomendada del PIA

```
/proyecto
  README.md
  main.py
  /src
    api_client.py
    validators.py
    cleaner.py
    utils.py
  /data
    /raw
    /clean
```

Regla de oro:

Todo lo que no sea código va en /data. Todo lo que sea lógica va en /src.

3. ¿Qué es un módulo en Python?

Un módulo es un archivo .py que contiene funciones.

Ejemplo:

validators.py

```
def validar_nombre(nombre):  
    return nombre.isalpha()
```

Importarlo:

```
from validators import validar_nombre
```

4. Cómo dividir un script grande en funciones pequeñas

Ejemplo de un script desordenado:

```
response = requests.get(url)  
if response.status_code == 200:  
    data = response.json()  
    nombre = data["name"].capitalize()  
    altura = int(data["height"])  
    # ...
```

Versión modular:

api_client.py

```
def get_data(url):  
    # conexión y errores
```

validators.py

```
def validar_nombre(nombre):  
    # regex o reglas
```

cleaner.py

```
def limpiar_datos(data):  
    # normalización
```

main.py

```
data = get_data(url)  
if validar_nombre(data["name"]):  
    clean = limpiar_datos(data)
```

5. Uso de os para crear carpetas (opcional)

```
import os  
  
os.makedirs("data/raw", exist_ok=True)  
os.makedirs("data/clean", exist_ok=True)
```

6. Uso de sys.argv para recibir parámetros

```
import sys  
  
print(sys.argv)  
Ejemplo de ejecución:  
python main.py ditto  
Salida:  
['main.py', 'ditto']
```

7. Buenas prácticas de modularidad

- Un archivo = una responsabilidad
- Funciones pequeñas
- No mezclar conexión con limpieza
- No duplicar código
- Documentar funciones
- Mantener nombres consistentes



EJEMPLOS GUÍA (Aplicables al PIA)

✓ Ejemplo 1: *api_client.py*

```
import requests

def get_data(url):
    try:
        response = requests.get(url)
        if response.status_code == 200:
            return response.json()
        else:
            print("Error en API:", response.status_code)
            return None
    except Exception as e:
        print("Error de conexión:", e)
        return None
```

✓ Ejemplo 2: *validators.py*

```
def validar_id(valor):
    return valor.isdigit()
```

✓ Ejemplo 3: *cleaner.py*

```
def limpiar_datos(data):
    clean = {
        "nombre": data["name"].capitalize(),
        "altura": int(data["height"]),
    }
    return clean
```

✓ Ejemplo 4: *utils.py*

```
def imprimir_banner(texto):
    print("="*40)
    print(texto)
    print("="*40)
```



ACTIVIDAD EN CLASE

Cada equipo debe:

1. Crear carpetas

```
data/  
data/raw/  
data/clean/  
src/
```

2. Crear archivos dentro de src/

- api_client.py
- validators.py
- cleaner.py
- utils.py

3. Agregar una función mínima en cada módulo

4. Probar imports

```
from utils import imprimir_banner  
imprimir_banner("Hola")
```

5. Crear main.py

6. Probar que main.py importe módulos

7. Crear carpetas automáticamente con os (opcional para aprender a usar el módulo)

8. Probar sys.argv

9. Subir todo a GitHub

10. Revisar que todos los integrantes aparezcan en los commits



TAREA (Enfocada al Avance 2 – 26 de abril)

Cada equipo debe:

✓ Integrar validación + limpieza en un solo flujo

Usar funciones de:

- validators.py
- cleaner.py

✓ Preparar api_client.py

Debe incluir:

- función GET
- manejo básico de errores

✓ Subir avances a GitHub

- Estructura completa
- Módulos creados
- Commits de todos